Real-world (a tool backed by a real API call) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `real_api_tool_practice.py`: a tool that only returns hardcoded data teaches you nothing about the failure modes real tools actually have — timeouts, bad responses, rate limits. Wrapping Doc02's `request_with_retry()` inside a `@tool` is what makes this real, not simulated. **If not:** the Build Task's "one tool that calls a real service" requirement would be the first time you ever put your Doc02 retry logic behind a tool, with no smaller version to trust it against.

All examples below need Doc02's `http_client.py`, `exceptions.py` and `logging_setup.py` copied, unchanged, into `practice/` next to this file.

Basic Version

Approach 1 — the direct way

```
# real_api_tool_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from http_client import request_with_retry  # your Doc02 client

WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

CITY_COORDINATES = {
    "lahore": {"latitude": 31.55, "longitude": 74.34},
    "karachi": {"latitude": 24.86, "longitude": 67.01},
}

@tool
def get_weather(city: str) -> str:
    """Get the current temperature for a named city."""
    coords = CITY_COORDINATES.get(city.lower())
    if coords is None:
        return f"No coordinates on file for {city}."

    params = {
        "latitude": coords["latitude"],
```

```
        "longitude": coords["longitude"],
        "current_weather": True,
    }
    # how: Doc02's signature — the HTTP method first, then the URL
    response = request_with_retry("GET", WEATHER_URL, params=params)
    temperature = response["current_weather"]["temperature"]
    return f"Currently {temperature}°C in {city}"

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
model_with_tools = model.bind_tools([get_weather])

response = model_with_tools.invoke("What's the weather in Lahore?")
print("Requested tool calls:", response.tool_calls)

for call in response.tool_calls:
    if call["name"] == "get_weather":
        result = get_weather.invoke(call["args"])
        print("Real result:", result)
```

Expected output:

```
Requested tool calls: [{'name': 'get_weather', 'args': {'city': 'Lahore'},
    'id': 'call_abc', 'type': 'tool_call'}]
Real result: Currently 34.2°C in Lahore
```

This version makes a real network call through your existing retry logic, and returns a plain text summary. It only knows 2 cities, and if the weather service fails after all retries, Doc02's error crashes the whole program — worth fixing next.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — params-building split out, and checked

```
# real_api_tool_practice.py
from langchain_core.runnables import Runnable
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

from exceptions import PermanentHTTPError, TransientHTTPError
from http_client import request_with_retry


WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

CITY_COORDINATES = {
    "lahore": {"latitude": 31.55, "longitude": 74.34},
    "karachi": {"latitude": 24.86, "longitude": 67.01},
}


def build_weather_params(city: str) -> dict | None:
    # why: pulled out on its own so "do we have coordinates for this city"
    # is testable directly, with no real network call or running model.
    coords = CITY_COORDINATES.get(city.lower())
    if coords is None:
        return None
    return {
        "latitude": coords["latitude"],
        "longitude": coords["longitude"],
        "current_weather": True,
    }


@tool
def get_weather(city: str) -> str:
    """Get the current temperature for a named city."""
    params = build_weather_params(city)
    if params is None:
        return f"No coordinates on file for {city}."
```

```
    try:
        # how: reuses Doc02's request_with_retry() — the tool never calls
        # requests directly, so it gets timeout/retry handling for free.
        response = request_with_retry("GET", WEATHER_URL, params=params)
    except (TransientHTTPError, PermanentHTTPError) as e:
        # why: Doc02's two named errors become a string the model can
        # read — the tool never crashes the program
        return f"Error: the weather service failed: {e}"

    temperature = response["current_weather"]["temperature"]
    return f"Currently {temperature}°C in {city}"


def get_model_with_tools() -> Runnable:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return model.bind_tools([get_weather])


def main() -> None:
    model_with_tools = get_model_with_tools()
    response = model_with_tools.invoke("What's the weather in Lahore?")

    if not response.tool_calls:
        print("The model answered directly, without calling a tool.")
        print(response.content)
        return

    for call in response.tool_calls:
        if call["name"] == "get_weather":
            result = get_weather.invoke(call["args"])
            print(f"Called get_weather({call['args']}) -> {result}")
        else:
            print(f"Unexpected tool requested: {call['name']}")


if __name__ == "__main__":
    main()
```

Expected output:

```
Called get_weather({'city': 'Lahore'}) -> Currently 34.2°C in Lahore
```

Difference from Basic: pulling `build_weather_params()` out as its own function makes the "do we have coordinates for this city" check independently testable, without needing a real network call or a running model. Catching Doc02's two named errors (`TransientHTTPError` , `PermanentHTTPError`) turns a failed service into a string the model can read, instead of a crash.

Checking `response.tool_calls` before looping over it also matters here for the same reason it mattered in the Basic exercise: an obvious-looking question is still not a *guarantee* the model calls anything.

Which one should you actually write? Intermediate's version is enough for this exercise — the plumbing (model → tool → `request_with_retry()` → real API) working end to end is the point. Two gaps are worth naming honestly rather than fixing here: `response["current_weather"]["temperature"]` assumes the API's response shape never changes, and `CITY_COORDINATES` only actually covers the 2 cities someone typed into it even though the tool's description promises "a named city." Neither needs fixing in this exercise — a `KeyError` here is the same category of crash the next exercise (`missing_argument_handling`) deliberately teaches you to catch, and a hardcoded lookup table is a fine demo shortcut as long as you can say out loud that it's one.